

Instituto Tecnológico de Aeronáutica

PO-249

**INTRODUÇÃO ÀS REDES NEURAIS E AOS GRADIENTES MODELOS DE
LINGUAGEM
ATIVIDADE 03**

Aluno: Luiz Eduardo T. C. Cartolano

Prof: Profº Dr. Mauri de Oliveira

São José dos Campos, SP - Brasil

2025

1 Introdução

A aprendizagem de redes neurais artificiais tem sido um dos pilares do avanço recente da inteligência artificial e do aprendizado de máquina. No centro desse processo está o método do *backpropagation*, ou retropropagação do erro, que possibilita o ajuste eficiente dos pesos sinápticos de uma rede neural por meio do cálculo do gradiente do erro em relação a cada peso da rede. Esse algoritmo, introduzido de forma prática na década de 1980, permitiu o treinamento de redes neurais com múltiplas camadas, superando as limitações impostas pelos métodos de aprendizagem anteriores, que eram restritos a arquiteturas de camada única.

O *backpropagation* baseia-se na aplicação da regra da cadeia do cálculo diferencial para propagar os erros da camada de saída em direção às camadas anteriores, atualizando os pesos de forma a minimizar uma função de custo. Esse processo iterativo de ajuste é realizado em conjunto com métodos de otimização, como o gradiente descendente, permitindo que a rede aprenda representações complexas a partir dos dados. Assim, o *backpropagation* constitui a espinha dorsal do aprendizado supervisionado em redes neurais profundas e continua sendo um componente essencial em inúmeras aplicações modernas, como reconhecimento de padrões, visão computacional e processamento de linguagem natural.

2 Conceitos Matemáticos

O método do *backpropagation* baseia-se em conceitos fundamentais do cálculo diferencial, em especial na aplicação da regra da cadeia. Esses conceitos permitem calcular como pequenas variações nas entradas de uma função afetam sua saída, possibilitando ajustar os parâmetros da rede neural para minimizar o erro.

2.1 Derivadas e Gradientes

A derivada de uma função [2] $f(x)$ em relação a uma variável x mede a taxa de variação de f em torno de x :

$$\frac{df(x)}{dx} = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}. \quad (1)$$

Em funções multivariáveis, usamos o conceito de *gradiente*, que é o vetor das derivadas parciais em relação a cada variável de entrada:

$$\nabla f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right) \quad (2)$$

2.2 Regra da Cadeia

A regra da cadeia [2] é um princípio fundamental para derivar funções compostas. Se $y = f(u)$ e $u = g(x)$, então:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}. \quad (3)$$

Em uma forma mais geral, para funções multivariáveis, se $y = f(u_1, u_2, \dots, u_n)$ e cada $u_i = g_i(x)$, temos:

$$\frac{dy}{dx} = \sum_{i=1}^n \frac{\partial f}{\partial u_i} \cdot \frac{du_i}{dx}. \quad (4)$$

Esse mecanismo de propagação das derivadas ao longo de funções compostas é a base do *backpropagation*, permitindo calcular os gradientes dos pesos em redes neurais.

3 Derivação do Backpropagation

Nesta seção apresentamos a derivação detalhada do algoritmo de retropropagação do erro [1], considerando uma rede com múltiplas unidades de saída. A notação e os passos são descritos de forma explícita para evidenciar a aplicação da regra da cadeia no cálculo dos gradientes.

3.1 Notação e Conceitos Básicos

Para entender a derivação do *backpropagation*, apresentamos a seguir a notação e os conceitos fundamentais usados ao longo do cálculo:

- **Camada de entrada:** cada neurônio de entrada é indexado por i e recebe como entrada o valor x_i da amostra.
- **Camada oculta:** cada neurônio oculto é indexado por j . A soma ponderada de suas entradas é

$$v_j = \sum_i w_{ji} x_i,$$

onde w_{ji} é o peso da conexão da entrada i para o neurônio j . A saída (ativação) do neurônio oculto é

$$\sigma(v_j) = \phi(v_j),$$

sendo ϕ a função de ativação da camada oculta. A derivada da ativação é $\phi'(v_j)$, utilizada na retropropagação.

- **Camada de saída:** consideramos múltiplos neurônios de saída, indexados por $k = 0, \dots, K$. A soma ponderada de cada neurônio de saída é

$$v_k = \sum_j w_{kj} \sigma(v_j),$$

e a saída da rede é

$$y_k = \varphi(v_k),$$

onde φ é a função de ativação da saída. A derivada é $\varphi'(v_k)$.

- **Valor desejado (target):** d_k é o valor que a rede deve produzir para a amostra atual no neurônio de saída k .
- **Taxa de aprendizagem:** $\eta > 0$ é um parâmetro que controla o tamanho dos ajustes nos pesos durante o treinamento.
- **Momento:** $\alpha \geq 0$ é a taxa de momento. Denotamos $\Delta w_{ji}^{(t-1)}, \Delta w_{kj}^{(t-1)}, \Delta b_j^{(t-1)}, \Delta b_k^{(t-1)}$ como variações dos pesos e biases na iteração anterior.

Esses elementos formam a base para o cálculo do gradiente do erro em relação aos pesos, que é realizado na etapa de *backpropagation*.

3.2 Derivação do gradiente para w_{kj}

Queremos calcular $\frac{\partial E}{\partial w_{kj}}$. Aplicamos a regra da cadeia diretamente sobre os termos que dependem de w_{kj} :

$$\frac{\partial E}{\partial w_{kj}} = \frac{\partial E}{\partial y_k} \cdot \frac{\partial y_k}{\partial v_k} \cdot \frac{\partial v_k}{\partial w_{kj}}. \quad (5)$$

Calculemos cada fator separadamente:

$$\frac{\partial E}{\partial y_k} = \frac{\partial}{\partial y_k} \left(\frac{1}{2} (d_k - y_k)^2 \right) = y_k - d_k \quad (6)$$

$$\frac{\partial y_k}{\partial v_k} = \varphi'(v_k) \quad (7)$$

$$\frac{\partial v_k}{\partial w_{kj}} = \sigma(v_j) \quad (8)$$

Multiplicando:

$$\frac{\partial E}{\partial w_{kj}} = (y_k - d_k) \varphi'(v_k) \sigma(v_j). \quad (9)$$

Agora usamos a regra do gradiente descendente (forma clássica):

$$\begin{aligned} w_{kj} &= w_{kj} - \eta \frac{\partial E}{\partial w_{kj}} \\ &= w_{kj} - \eta (y_k - d_k) \varphi'(\mathbf{v}_k) \sigma(v_j). \end{aligned} \quad (10)$$

É conveniente definir o *delta* da saída com sinal ($d_k - y_k$) para tornar a atualização numa soma:

$$\delta_k = (d_k - y_k) \varphi'(\mathbf{v}_k) \quad (11)$$

com essa definição a atualização escreve-se de forma compacta:

$$w_{kj} = w_{kj} + \eta \delta_k \sigma(v_j) \quad (12)$$

Adicionando um termo de momento, temos:

$$w_{kj} = w_{kj} + \eta \delta_k \sigma(v_j) + \alpha \Delta w_{kj}^{(t-1)} \quad (13)$$

3.3 Derivação do gradiente para w_{ji}

Agora o peso w_{ji} afeta E indiretamente via $\sigma(v_j)$ e todos os y_k . Aplicamos a regra da cadeia:

$$\frac{\partial E}{\partial w_{ji}} = \sum_{k=0}^K \frac{\partial E}{\partial y_k} \frac{\partial y_k}{\partial \mathbf{v}_k} \frac{\partial \mathbf{v}_k}{\partial \sigma(v_j)} \frac{\partial \sigma(v_j)}{\partial \mathbf{v}_j} \frac{\partial \mathbf{v}_j}{\partial w_{ji}}. \quad (14)$$

Calculamos cada fator (alguns já vistos):

$$\frac{\partial E}{\partial y_k} = y_k - d_k \quad (15)$$

$$\frac{\partial y_k}{\partial \mathbf{v}_k} = \varphi'(\mathbf{v}_k) \quad (16)$$

$$\frac{\partial \mathbf{v}_k}{\partial \sigma(v_j)} = w_{kj} \quad (17)$$

$$\frac{\partial \sigma(v_j)}{\partial \mathbf{v}_j} = \phi'(\mathbf{v}_j) \quad (18)$$

$$\frac{\partial \mathbf{v}_j}{\partial w_{ji}} = x_i \quad (19)$$

Multiplicando todos:

$$\frac{\partial E}{\partial w_{ji}} = \sum_{k=1}^K (y_k - d_k) \varphi'(\mathbf{v}_k) w_{kj} \phi'(\mathbf{v}_j) x_i. \quad (20)$$

Usando a definição do δ_k da Equação 11, notamos que

$$(y_k - d_k) \varphi'(\mathbf{v}_k) = -\delta_k. \quad (21)$$

Logo

$$\frac{\partial E}{\partial w_{ji}} = - \left(\sum_{k=0}^K w_{kj} \delta_k \right) \phi'(v_j) x_i. \quad (22)$$

É prático definir o *delta* da unidade oculta j como

$$\delta_j = \phi'(v_j) \sum_{k=0}^K w_{kj} \delta_k \quad (23)$$

Com essa definição,

$$\frac{\partial E}{\partial w_{ji}} = -\delta_j x_i. \quad (24)$$

Aplicando gradiente descendente:

$$\begin{aligned} w_{ji} &= w_{ji} - \eta \frac{\partial E}{\partial w_{ji}} \\ &= w_{ji} + \eta \delta_j x_i. \end{aligned} \quad (25)$$

Adicionando um termo de momento, temos:

$$w_{ji} = w_{ji} + \eta \delta_j x_i + \alpha \Delta w_{ji}^{(t-1)} \quad (26)$$

3.4 Atualizações usando função sigmoide em todas as camadas

Se adotarmos a função sigmoide

$$\sigma(v) = \frac{1}{1 + e^{-v}} \quad (27)$$

para as camadas oculta e de saída, a derivada é

$$\sigma'(v) = \sigma(v) (1 - \sigma(v)). \quad (28)$$

Logo, os δ ficam:

$$\delta_k = (d_k - y_k) \sigma'(v_k), \quad (29)$$

e

$$\delta_j = \sigma'(v_j) \sum_{k=0}^K w_{kj} \delta_k. \quad (30)$$

As atualizações de pesos são:

$$w_{kj} = w_{kj} + \eta \delta_k \sigma(v_j) + \alpha \Delta w_{kj}^{(t-1)} \quad (31)$$

e

$$w_{ji} = w_{ji} + \eta \delta_j x_i + \alpha \Delta w_{ji}^{(t-1)} \quad (32)$$

3.5 Atualizações usando ReLU na camada oculta e sigmoide na camada de saída

Se a camada oculta usar ReLU

$$\text{ReLU}(v) = \max(0, v)$$

com derivada

$$\text{ReLU}'(v) = \begin{cases} 1, & v > 0 \\ 0, & v \leq 0 \end{cases},$$

e a saída continuar com sigmoide $\sigma(v)$, então:

$$\delta_k = (d_k - y_k) \sigma'(v_k) \quad (33)$$

e

$$\delta_j = \text{ReLU}'(v_j) \sum_{k=0}^K w_{kj} \delta_k \quad (34)$$

As atualizações de pesos permanecem:

$$w_{kj} = w_{kj} + \eta \delta_k \text{ReLU}(v_j) + \alpha \Delta w_{kj}^{(t-1)} \quad (35)$$

e

$$w_{ji} = w_{ji} + \eta \delta_j x_i + \alpha \Delta w_{ji}^{(t-1)} \quad (36)$$

4 Exemplo de Retropropagação

4.1 Parâmetros e Inicializações

$$\eta = 0,5, \quad \alpha = 0,1, \quad \phi(z) = \varphi(z) = \sigma(z) = \frac{1}{1 + e^{-z}} \quad (37)$$

$$\text{Pesos iniciais: } \forall w = 0,1, \quad \text{bias: } b_j = b_k = 0,1, \quad x_0 = 1 \quad (38)$$

4.2 Amostra 1 — Maçã

$$x = \begin{bmatrix} 1 \\ 0,5 \\ 0,2 \end{bmatrix}, \quad d = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad (39)$$

Forward — Camada Oculta

$$v_1 = w_{10} x_0 + w_{11} x_1 + w_{12} x_2 \approx 0,170, \quad \sigma(v_1) \approx 0,542 \quad (40)$$

$$v_2 = w_{20} x_0 + w_{21} x_1 + w_{22} x_2 \approx 0,170, \quad \sigma(v_2) \approx 0,542 \quad (41)$$

Forward — Camada de Saída

$$v_1^{(o)} = w_{10}^{(o)} x_0 + w_{11}^{(o)} \sigma(v_1) + w_{12}^{(o)} \sigma(v_2) \approx 0,208, \quad y_1 = \varphi(v_1^{(o)}) \approx 0,552 \quad (42)$$

$$v_2^{(o)} = w_{20}^{(o)} x_0 + w_{21}^{(o)} \sigma(v_1) + w_{22}^{(o)} \sigma(v_2) \approx 0,208, \quad y_2 = \varphi(v_2^{(o)}) \approx 0,552 \quad (43)$$

Erro

$$E = \frac{1}{2} \sum_k (d_k - y_k)^2 \approx 0,253 \quad (44)$$

Backward — Saída

$$\delta_1 = (d_1 - y_1) \varphi'(v_1^{(o)}) \approx 0,111 \quad (45)$$

$$\delta_2 = (d_2 - y_2) \varphi'(v_2^{(o)}) \approx -0,136 \quad (46)$$

Backward — Oculta

$$\delta_1^{(h)} = \phi'(v_1) \left(w_{11}^{(o)} \delta_1 + w_{21}^{(o)} \delta_2 \right) \approx -0,001 \quad (47)$$

$$\delta_2^{(h)} = \phi'(v_2) \left(w_{12}^{(o)} \delta_1 + w_{22}^{(o)} \delta_2 \right) \approx -0,001 \quad (48)$$

Atualização dos Pesos — Sem Momento

$$w_{1j}^{(o)} \leftarrow w_{1j}^{(o)} + \eta \delta_1 \cdot \sigma(v_j) \quad (49)$$

$$w_{2j}^{(o)} \leftarrow w_{2j}^{(o)} + \eta \delta_2 \cdot \sigma(v_j) \quad (50)$$

$$w_{ji} \leftarrow w_{ji} + \eta \delta_j^{(h)} \cdot x_i \quad (51)$$

Pesos após a Amostra 1 (Maçã), arredondados:

$$w_h \approx \begin{bmatrix} 0,099 & 0,100 & 0,100 \\ 0,099 & 0,100 & 0,100 \end{bmatrix}, \quad w_o \approx \begin{bmatrix} 0,156 & 0,130 & 0,130 \\ 0,032 & 0,063 & 0,063 \end{bmatrix} \quad (52)$$

4.3 Amostra 2 — Laranja

$$x = \begin{bmatrix} 1 \\ 0,8 \\ 0,6 \end{bmatrix}, \quad d = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad (53)$$

Forward — Camada Oculta

$$v_1 = w_{10} x_0 + w_{11} x_1 + w_{12} x_2 \approx 0,240, \quad \sigma(v_1) \approx 0,560 \quad (54)$$

$$v_2 = w_{20} x_0 + w_{21} x_1 + w_{22} x_2 \approx 0,240, \quad \sigma(v_2) \approx 0,560 \quad (55)$$

Forward — Camada de Saída

$$v_1^{(o)} = w_{10}^{(o)} x_0 + w_{11}^{(o)} \sigma(v_1) + w_{12}^{(o)} \sigma(v_2) \approx 0,301, \quad y_1 = \varphi(v_1^{(o)}) \approx 0,575 \quad (56)$$

$$v_2^{(o)} = w_{20}^{(o)} x_0 + w_{21}^{(o)} \sigma(v_1) + w_{22}^{(o)} \sigma(v_2) \approx 0,102, \quad y_2 = \varphi(v_2^{(o)}) \approx 0,526 \quad (57)$$

Erro

$$E = \frac{1}{2} \sum_k (d_k - y_k)^2 \approx 0,278 \quad (58)$$

Backward — Saída

$$\delta_1 = (d_1 - y_1) \varphi'(v_1^{(o)}) \approx -0,140 \quad (59)$$

$$\delta_2 = (d_2 - y_2) \varphi'(v_2^{(o)}) \approx 0,118 \quad (60)$$

Backward — Oculta

$$\delta_1^{(h)} = \phi'(v_1) \left(w_{11}^{(o)} \delta_1 + w_{21}^{(o)} \delta_2 \right) \approx -0,003 \quad (61)$$

$$\delta_2^{(h)} = \phi'(v_2) \left(w_{12}^{(o)} \delta_1 + w_{22}^{(o)} \delta_2 \right) \approx -0,003 \quad (62)$$

Atualização dos Pesos — Com Momento

$$w_{ji} \leftarrow w_{ji} + \eta \delta_j^{(h)} x_i + \alpha \Delta w_{ji}^{(\text{ant})} \quad (63)$$

$$w_{kj}^{(o)} \leftarrow w_{kj}^{(o)} + \eta \delta_k \sigma(v_j) + \alpha \Delta w_{kj}^{(o, \text{ant})} \quad (64)$$

Pesos finais após a primeira época (Maçã + Laranja), arredondados:

$$w_h \approx \begin{bmatrix} 0,098 & 0,099 & 0,099 \\ 0,098 & 0,099 & 0,099 \end{bmatrix}, \quad w_o \approx \begin{bmatrix} 0,091 & 0,094 & 0,094 \\ 0,084 & 0,092 & 0,092 \end{bmatrix} \quad (65)$$

5 Experimento

Para avaliar o comportamento do algoritmo de retropropagação em diferentes funções de ativação, foi construído um Perceptron Multicamadas (MLP) com arquitetura (2, 2, 2), conforme ilustrado na Figura 1.

As entradas da rede representam duas características das frutas: (i) tamanho, sendo $x_1 = 0,5$ para Maçã e $x_1 = 0,8$ para Laranja; e (ii) textura, sendo $x_2 = 0,2$ para Maçã e $x_2 = 0,6$ para Laranja. A saída da rede é codificada em um vetor one-hot de duas posições: $[1, 0]$ para Maçã e $[0, 1]$ para Laranja.

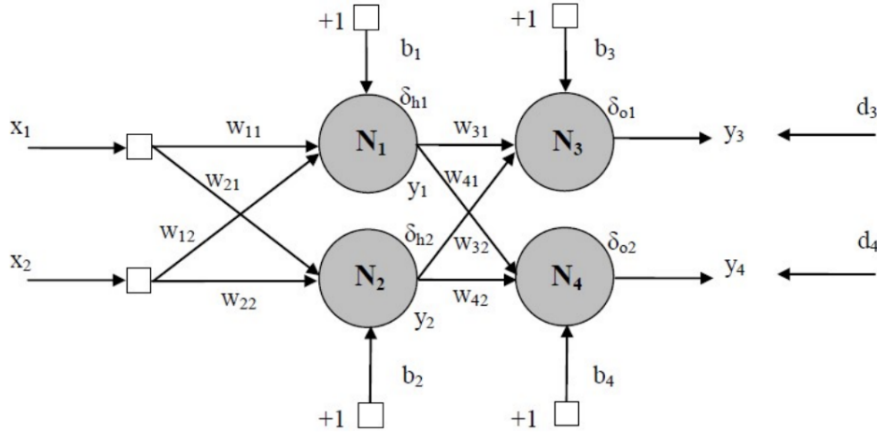


Figura 1: Arquitetura do MLP (2, 2, 2) utilizado nos experimentos.

Foram conduzidos dois experimentos distintos, utilizando os mesmos hiperparâmetros e conjunto de dados, alterando apenas a função de ativação:

1. Função sigmoide em todas as camadas.
2. Função ReLU na camada oculta e função sigmoide na camada de saída.

Em ambos os casos, os pesos e *bias* foram inicializados com 0,1. O treinamento foi realizado utilizando uma taxa de aprendizado $\eta = 0,5$ e uma taxa de momento $\alpha = 0,1$, durante 1000 épocas. A função de custo utilizada para a otimização foi a raiz do erro quadrático médio (RMSE).

O objetivo é comparar a convergência e a capacidade de separação das classes entre as duas configurações de funções de ativação.

6 Resultados

A Tabela 1 apresenta um resumo do desempenho obtido nos experimentos descritos na Seção 5. Observa-se que a configuração com Sigmóide na camada de entrada e ReLU na camada oculta obteve um erro final (*loss*) de 0,0318, significativamente menor em relação à configuração totalmente sigmóide (0,0846). Esse resultado indica que a utilização da função ReLU na camada oculta favorece uma convergência mais eficiente e um melhor ajuste aos dados de treinamento.

Tabela 1: Desempenho das duas configurações testadas para a MLP (2–2–2) com os novos experimentos.

Configuração	Épocas	Tempo (s)	Loss Final
Sigmóide + ReLU	1000	0,0226	0,0318
Sigmóide + Sigmóide	1000	0,0241	0,0846

As Figuras 2 e 3 mostram a evolução da função de perda (*loss*) ao longo das 1000 épocas para cada configuração. Nota-se que ambas as redes convergem, porém a configuração Sigmóide + ReLU atinge valores de perda final menores e apresenta uma convergência mais estável do que a configuração Sigmóide + Sigmóide. Isso sugere que a função ReLU na camada oculta melhora a propagação do gradiente, acelerando o aprendizado e contribuindo para uma modelagem mais precisa dos dados.

7 Discussão

Os resultados obtidos indicam que a escolha da função de ativação na camada oculta tem um impacto significativo no desempenho da MLP. A configuração Sigmóide + ReLU apresentou uma perda final menor (0,0318) em comparação à configuração totalmente sigmóide (0,0846), além de convergência mais rápida e estável. Isso sugere que a ReLU facilita a propagação do gradiente, evitando problemas de saturação que podem ocorrer com a função sigmóide e que retardam o aprendizado nas camadas iniciais.

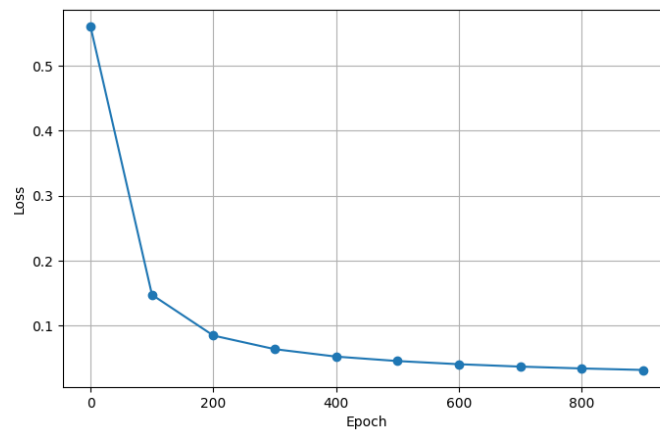


Figura 2: Evolução da função de perda ($loss$) para a configuração Sigmóide (camada de entrada) + ReLU (camada oculta).

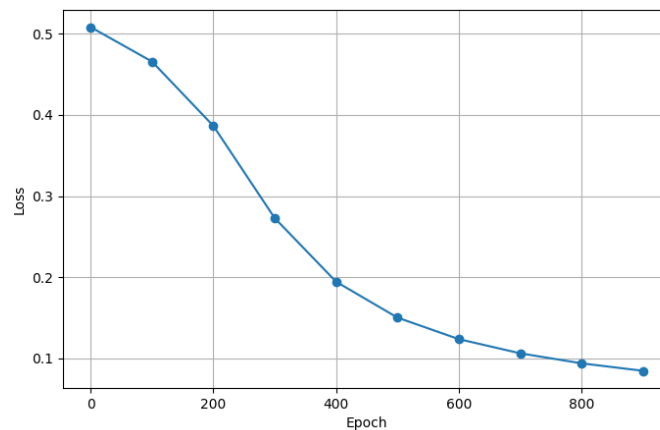


Figura 3: Evolução da função de perda ($loss$) para a configuração Sigmóide em ambas as camadas.

Além da diferença na perda final, os pesos e *bias* obtidos após o treinamento refletem a capacidade de ajuste da rede. A configuração Sigmóide + ReLU mostra pesos de magnitude mais moderada e balanceada, indicando que a rede consegue distribuir melhor o aprendizado entre os neurônios, enquanto a configuração totalmente sigmóide apresenta pesos mais extremos, possivelmente decorrentes de ajustes compensatórios para superar a saturação da função de ativação.

Em termos de tempo de treinamento, ambas as configurações são comparáveis, com pequenas diferenças que não impactam significativamente o desempenho. Dessa forma, a escolha da função de ativação adequada se mostra mais determinante para a qualidade do modelo do que variações pequenas no tempo computacional.

8 Conclusão

Neste estudo, avaliou-se o desempenho de diferentes combinações de funções de ativação em uma MLP (2–2–2) aplicada a dados de treinamento específicos. Os experimentos demonstraram que a utilização da função ReLU na camada oculta, combinada com sigmóide na camada de saída, resulta em melhor desempenho em termos de perda final, estabilidade e velocidade de convergência, em comparação à configuração totalmente sigmóide.

Os resultados reforçam a importância da escolha adequada das funções de ativação, mostrando que a ReLU pode melhorar significativamente a capacidade de aprendizado da rede, especialmente em camadas ocultas, ao mitigar problemas de saturação e facilitar a propagação do gradiente. Esse estudo evidencia que pequenas alterações na arquitetura e nas funções de ativação podem gerar diferenças expressivas no desempenho final de redes neurais.

Referências

- [1] Simon S. Haykin. *Redes Neurais: Princípios e Prática*. Bookman, Porto Alegre, 2. ed. edition, 2001.
- [2] James Stewart. *Cálculo – Volume 1*. Cengage Learning, São Paulo, 7 edition, 2012.